

# PageRank su Architetture Ibride

Parallelizzazione con OpenMP e CUDA

Emanuele Cecchia

Corso di Programmazione su Architetture Parallele

A.A. 2025–26 — Prof. Andrea Formisano

12 settembre 2026

## Abstract

Il progetto implementa e analizza l'algoritmo PageRank su architetture parallele, confrontando tre versioni: una sequenziale, che fa da riferimento per ogni misura di speed-up, una puramente multicore basata su OpenMP e una ibrida che distribuisce il carico fra CPU e GPU tramite CUDA. Sequenziale e OpenMP condividono lo stesso file sorgente e differiscono solo per le opzioni di compilazione, così che il riferimento non possa divergere dal codice parallelo né essere falsato dai costi di gestione dei thread. Il grafo è memorizzato trasposto in formato CSR, scelta che rende il nucleo di calcolo privo di scritture concorrenti e quindi parallelizzabile senza sincronizzazione. La correttezza è verificata contro tre implementazioni indipendenti. Le misure sulla versione OpenMP mostrano uno speed-up di  $2,4\text{--}2,9\times$  su 4 core fisici e, soprattutto, una saturazione oltre i 4 thread che identifica il calcolo come *memory-bound*: è questo il dato che motiva quantitativamente il passaggio alla GPU.

**[DA COMPLETARE]** Aggiornare l'abstract con i risultati CUDA una volta completata la fase ibrida.

## Indice

|          |                                                |          |
|----------|------------------------------------------------|----------|
| <b>1</b> | <b>Descrizione del problema affrontato</b>     | <b>3</b> |
| 1.1      | Il problema . . . . .                          | 3        |
| 1.2      | Damping factor e Dangling Mass . . . . .       | 3        |
| 1.3      | Forma matriciale . . . . .                     | 4        |
| 1.4      | Algoritmo . . . . .                            | 4        |
| 1.5      | Necessità della parallelizzazione . . . . .    | 5        |
| <b>2</b> | <b>Metodologia adottata per la soluzione</b>   | <b>5</b> |
| 2.1      | Rappresentazione dei dati: CSR . . . . .       | 5        |
| 2.2      | Formato binario e pre-elaborazione . . . . .   | 7        |
| 2.3      | Struttura del codice . . . . .                 | 8        |
| 2.4      | Validazione del file in ingresso . . . . .     | 8        |
| 2.5      | Implementazione sequenziale e OpenMP . . . . . | 9        |
| 2.6      | Precisione: float contro double . . . . .      | 9        |
| 2.7      | Implementazione ibrida OpenMP + CUDA . . . . . | 10       |
| 2.8      | Strategia di verifica . . . . .                | 10       |

|          |                                                                              |           |
|----------|------------------------------------------------------------------------------|-----------|
| <b>3</b> | <b>Risultati sperimentali</b>                                                | <b>11</b> |
| 3.1      | Ambiente di prova . . . . .                                                  | 11        |
| 3.2      | Istanze di prova . . . . .                                                   | 11        |
| 3.3      | Correttezza . . . . .                                                        | 11        |
| 3.4      | Scalabilità della versione OpenMP . . . . .                                  | 12        |
| 3.5      | Confronto fra le varianti <code>float</code> e <code>double</code> . . . . . | 13        |
| 3.6      | Risultati della versione ibrida . . . . .                                    | 13        |
| <b>4</b> | <b>Valutazioni e osservazioni</b>                                            | <b>13</b> |
| 4.1      | Punti di forza . . . . .                                                     | 13        |
| 4.2      | Punti deboli e limiti . . . . .                                              | 13        |
| 4.3      | Possibili miglioramenti . . . . .                                            | 14        |
| <b>5</b> | <b>Guida all'uso e riproducibilità</b>                                       | <b>14</b> |
| 5.1      | Requisiti . . . . .                                                          | 14        |
| 5.2      | Compilazione . . . . .                                                       | 14        |
| 5.3      | Preparazione dei dati . . . . .                                              | 14        |
| 5.4      | Esecuzione . . . . .                                                         | 15        |
| 5.5      | Verifica della correttezza . . . . .                                         | 15        |
| 5.6      | Riproduzione degli esperimenti . . . . .                                     | 15        |
| 5.7      | Codice sorgente . . . . .                                                    | 15        |

# 1 Descrizione del problema affrontato

## 1.1 Il problema

PageRank assegna a ogni nodo di un grafo orientato un punteggio di importanza, definito ricorsivamente: un nodo è importante se nodi importanti puntano ad esso. Nato per ordinare le pagine web in base alla struttura dei collegamenti ipertestuali [1], viene oggi applicato anche a reti sociali, reti di citazioni e reti biologiche.

La definizione è ricorsiva e si risolve iterativamente: si parte da una distribuzione uniforme e la si raffina finché non cambia più — in pratica, finché la variazione non scende sotto una soglia prefissata, oppure fino a un numero massimo di iterazioni. Equivale al modello del *navigatore casuale*, che segue collegamenti scelti a caso: il PageRank è la frazione di tempo che costui trascorre su ciascuna pagina. Ne discendono due proprietà della formula — il totale vale 1, perché il navigatore è sempre da qualche parte, e un collegamento pesa tanto meno quanti più ne partono dalla stessa pagina.

Formalmente, dato un grafo orientato  $G = (V, E)$  con  $N = |V|$ ,

$$\text{PR}_{t+1}(v) = \frac{1-d}{N} + d \left( \sum_{u: u \rightarrow v} \frac{\text{PR}_t(u)}{\text{outdeg}(u)} + \frac{\text{dangling}_t}{N} \right) \quad (1)$$

dove i simboli hanno il seguente significato:

|                     |                                                                                                                                                |
|---------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| $\text{PR}_t(v)$    | <i>PageRank</i> : punteggio del nodo $v$ alla $t$ -esima iterazione                                                                            |
| $\text{outdeg}(u)$  | numero di archi uscenti da $u$                                                                                                                 |
| $d$                 | <i>damping factor</i> , convenzionalmente 0,85                                                                                                 |
| $\text{dangling}_t$ | <i>dangling mass</i> : punteggio complessivo dei nodi senza archi uscenti<br>$\text{dangling}_t = \sum_{u: \text{outdeg}(u)=0} \text{PR}_t(u)$ |
| $N$                 | numero di nodi                                                                                                                                 |

## 1.2 Damping factor e Dangling Mass

Il termine  $(1-d)/N$ : un gruppo di pagine che si citano solo fra loro assorbirebbe, iterazione dopo iterazione, tutto il punteggio del grafo, perché i collegamenti vi entrano e nessuno ne esce. Con probabilità  $1-d$  il navigatore abbandona i collegamenti e salta su una pagina qualsiasi: questo impedisce l'accumulo e rende ogni nodo raggiungibile da ogni altro. È anche l'unica via d'uscita da un nodo senza archi uscenti, dove con  $d=1$  il navigatore resterebbe intrappolato per sempre.

Il termine  $\text{dangling}_t/N$ : un nodo senza archi uscenti non ha modo di ridistribuire il proprio rank. Il salto casuale lo libera con probabilità  $1-d$ , ma la quota  $d$  restante cercherebbe un collegamento che non esiste: è questa che il termine raccoglie e ridistribuisce. Ignorandolo, ad ogni iterazione una frazione della massa totale sparirebbe e il vettore cesserebbe di essere una distribuzione di probabilità. Nei dataset utilizzati questo non è un caso limite: il 15,6% dei nodi di **web-Google** e l'11,1% di **soc-LiveJournal1** sono *dangling* (Tabella 2). La conseguenza pratica è che  $\sum_v \text{PR}(v) = 1$  costituisce un test di correttezza.

Con entrambi i termini l'iterazione descrive una catena di Markov sui nodi del grafo: da ogni nodo — cicli chiusi e vicoli ciechi compresi — si può sempre raggiungere qualunque altro. Questo ci garantisce un'unica distribuzione stazionaria.

### 1.3 Forma matriciale

La stessa iterazione si scrive in forma vettoriale. Sia  $\mathbf{MT} \in \mathbb{R}^{N \times N}$  la *matrice di transizione* definita da

$$\mathbf{MT}_{vu} = \begin{cases} 1/\text{outdeg}(u) & \text{se esiste l'arco } u \rightarrow v, \\ 0 & \text{altrimenti,} \end{cases} \quad (2)$$

e sia  $\mathbf{1}$  il vettore di tutti uni. Detto  $\mathbf{PR}_t$  il vettore dei punteggi ( $\text{PR}(1), \dots, \text{PR}(N)$ ), l'iterazione (1) diventa

$$\mathbf{PR}_{t+1} = \frac{1-d}{N} \mathbf{1} + d \left( \mathbf{MT} \mathbf{PR}_t + \frac{\text{dangling}_t}{N} \mathbf{1} \right), \quad \text{dangling}_t = \mathbf{w}^\top \mathbf{PR}_t \quad (3)$$

dove  $\mathbf{w}$  è il vettore indicatore dei nodi dangling ( $w_u = 1$  se  $\text{outdeg}(u) = 0$ , altrimenti 0).

Il prodotto  $\mathbf{MT} \mathbf{PR}_t$  che compare nella (3) è una moltiplicazione matrice-sparsa per vettore (SpMV, Sparse Matrix-Vector Multiplication): una matrice sparsa moltiplica un vettore denso e ne produce uno nuovo, sempre denso, nella forma generica  $\mathbf{y} = \mathbf{Ax}$ . È questa l'operazione che il progetto parallelizza. Per non pagare gli zeri né in spazio né in tempo si memorizzano i soli elementi non nulli di  $\mathbf{MT}$ , usando formati come CSR (Compressed Sparse Row), che vedremo nella Sezione 2.1.

### 1.4 Algoritmo

---

**Algorithm 1** PageRank per iterazione di potenza (formulazione *pull*)

---

```

1:  $\text{PR}[v] \leftarrow 1/N$  per ogni  $v \in V$ 
2: repeat
3:    $\text{dangling} \leftarrow 0$ 
4:   for all  $u \in V$  do
5:     if  $\text{outdeg}[u] = 0$  then
6:        $\text{contrib}[u] \leftarrow 0$ ;  $\text{dangling} \leftarrow \text{dangling} + \text{PR}[u]$ 
7:     else
8:        $\text{contrib}[u] \leftarrow \text{PR}[u]/\text{outdeg}[u]$ 
9:     end if
10:  end for
11:   $\text{base} \leftarrow (1-d)/N + d \cdot \text{dangling}/N$ 
12:  for all  $v \in V$  do
13:     $\text{PR}'[v] \leftarrow \text{base} + d \sum_{u \rightarrow v} \text{contrib}[u]$ 
14:  end for
15:   $\varepsilon \leftarrow \sum_v |\text{PR}'[v] - \text{PR}[v]|$ 
16:  scambia  $\text{PR}$  e  $\text{PR}'$ 
17: until  $\varepsilon < \text{tolleranza}$ 

```

---

|               |                                                                                                                           |
|---------------|---------------------------------------------------------------------------------------------------------------------------|
| $N, d$        | numero di nodi e damping factor: scalari, costanti per tutta l'esecuzione                                                 |
| $PR[v]$       | punteggio corrente del nodo $v$ ; $PR$ è un vettore di $N$ componenti, una per nodo                                       |
| $PR'[v]$      | punteggio di $v$ calcolato nell'iterazione in corso; vettore di appoggio, della stessa forma di $PR$                      |
| $contrib[u]$  | contributo che $u$ manda lungo ciascuno dei suoi archi uscenti; vettore di $N$ componenti, ricalcolato ad ogni iterazione |
| $outdeg[u]$   | numero di archi uscenti da $u$ ; vettore di $N$ componenti, proprietà fissa del grafo                                     |
| $dangling$    | <i>dangling mass</i> : scalare, punteggio complessivo dei nodi senza archi uscenti                                        |
| $base$        | scalare, quota di punteggio che spetta a ogni nodo indipendentemente dagli archi                                          |
| $\varepsilon$ | scalare, variazione complessiva fra un'iterazione e la successiva                                                         |

Ogni iterazione ricostruisce il vettore dei punteggi a partire da quello precedente, con due passaggi sui nodi. Nel primo ogni nodo è visto come sorgente: si calcola quanto manda lungo ciascun arco uscente e si raccoglie al tempo stesso la *dangling mass*. Nel secondo ogni nodo è visto come destinazione: si sommano i contributi che arrivano dai suoi predecessori. Si ripete finché la variazione complessiva non scende sotto la tolleranza. Il ciclo è il *metodo delle potenze*: si rimoltiplica il vettore finché non si stabilizza, e non serve normalizzarlo a ogni passo perché l'iterazione conserva da sé la somma a 1. *Pull* indica invece che ogni nodo legge dai predecessori anziché scrivere nei successori, scelta motivata nella Sezione 2.1.

Esplicitando *base*, il calcolo di  $PR'[v]$  è esattamente l'iterazione (1):

$$PR'[v] = \underbrace{\frac{1-d}{N} + d \frac{dangling}{N}}_{base} + d \sum_{u: u \rightarrow v} contrib[u]$$

dove  $contrib[u] = PR[u]/outdeg[u]$  per i nodi con archi uscenti, e  $contrib[u] = 0$  per i *dangling*, il cui punteggio confluisce invece in *dangling*.

Il calcolo di  $contrib[u]$  è separato dal ciclo di accumulo per una ragione di costo: eseguito così richiede  $N$  divisioni per iterazione anziché  $M = |E|$ , e nei grafi considerati  $M \gg N$  (su `soc-LiveJournal1`, 69 milioni contro 4,8 milioni). La divisione è fra le operazioni aritmetiche più lente, quindi il risparmio è reale.

## 1.5 Necessità della parallelizzazione

La difficoltà del problema è interamente di scala. La matrice **MT** della Sezione 1.3 è *sparsa*: delle sue  $N^2$  celle solo  $M$  sono diverse da zero, e nei grafi reali  $M \ll N^2$ . Rappresentarla come matrice densa  $N \times N$  — una cella per ogni coppia  $(v, u)$  — richiederebbe una memoria quadratica in  $N$  e occupata quasi per intero da zeri, oltre a  $N^2$  moltiplicazioni per iterazione anziché  $M$ . Si adotta perciò una rappresentazione *sparsa*, che conservi i soli elementi non nulli — il formato **CSR** della Sezione 2.1. Restano comunque  $M$  archi da percorrere a ogni iterazione, e le iterazioni si susseguono fino a convergenza: questo rende necessario distribuire il calcolo su più unità di elaborazione.

## 2 Metodologia adottata per la soluzione

### 2.1 Rappresentazione dei dati: CSR

Per rappresentare i soli elementi non nulli della matrice **MT**, il formato **CSR** non usa una matrice bidimensionale  $N \times N$  ma tre array monodimensionali:

- `col_idx[M]`: gli indici di colonna dei non nulli, letti riga per riga;
- `row_ptr[N+1]`: `row_ptr[v]` è la posizione, dentro `col_idx`, in cui comincia la riga  $v$ ;
- `out_deg[N]`: il grado uscente di ogni nodo, cioè il denominatore  $\text{outdeg}(u)$  della (1).

La Figura 1 segue la costruzione su un grafo di cinque nodi e otto archi, dal grafo alla matrice agli array.

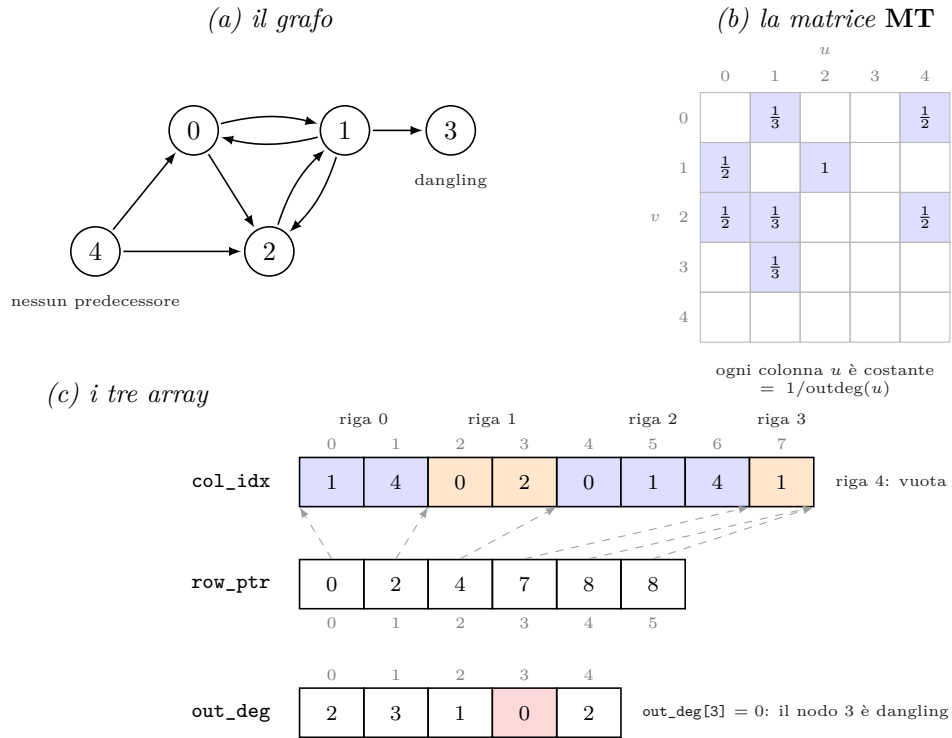


Figura 1: Costruzione del CSR del grafo trasposto, su un esempio con  $N = 5$  nodi e  $M = 8$  archi. In (b) la riga  $v$  della matrice  $\mathbf{MT}$  raccoglie i *predecessori* di  $v$ , e ogni colonna  $u$  ha il valore costante  $1/\text{outdeg}(u)$ . In (c) `col_idx` concatena le righe di (b), `row_ptr[v]` indica dove ciascuna comincia e `out_deg` conserva i denominatori. Le due condizioni limite sono distinte e vivono su array distinti: la riga 4 è vuota perché nessuno punta al nodo 4, mentre `out_deg[3]` è nullo perché il nodo 3 non ha archi uscenti.

Tutto ruota attorno a `row_ptr`: gli elementi della riga  $v$  sono `col_idx[row_ptr[v] ... row_ptr[v+1]-1]`, e la lunghezza della riga è la differenza fra due offset consecutivi, che quindi non va memorizzata a parte. Nell'esempio della figura la riga 2 comincia in posizione `row_ptr[2] = 4` e finisce prima di `row_ptr[3] = 7`: i predecessori del nodo 2 sono dunque `col_idx[4]`, `col_idx[5]` e `col_idx[6]`, cioè i nodi 0, 1 e 4 — come si può constatare nel grafo. Gli offset sono  $N + 1$  e non  $N$  proprio per questo motivo: la sentinella `row_ptr[N] = M` fa sì che l'ultima riga si tratti come tutte le altre.

**Perché manca l'array dei valori.** Un CSR generico ha un quarto array, `val[M]`, con il valore di ogni elemento non nullo. Qui non serve, e la Figura 1(b) mostra il perché: tutti i non nulli della colonna  $u$  valgono  $1/\text{outdeg}(u)$ , così che il valore dipenda dalla sola colonna e sia ricavabile da `out_deg`. Il vettore `contrib` dell'Algoritmo 1 porta l'osservazione fino in fondo applicando quella divisione una volta per nodo: il ciclo interno non contiene allora né divisioni né letture di valori, ma solo un'indirizione e una somma. Il risparmio non è trascurabile — su

soc-LiveJournal1 un `val[M]` in doppia precisione occuperebbe  $8M \approx 552$  MB, più di tutto il resto della struttura messo insieme.

**Perché il trasposto.** La scelta progettuale più importante del progetto è *quale* indice diventa la riga. L'equazione (1) richiede, per calcolare  $PR_{t+1}(v)$ , l'insieme dei nodi che puntano a  $v$ . Esistono due modi di organizzare il calcolo:

**Push** si scorre l'insieme degli archi  $u \rightarrow v$  e si somma il contributo di  $u$  dentro  $PR'(v)$ . Più archi convergono sullo stesso  $v$ , quindi più thread scrivono sulla stessa cella: si genera una *race condition* che richiede operazioni atomiche o lock.

**Pull** si scorre l'insieme dei nodi  $v$ , si leggono i suoi predecessori e si scrive  $PR'(v)$  una sola volta. Ogni nodo è di competenza di un solo thread: **non esiste alcuna scrittura concorrente**.

È stata adottata la formulazione *pull*, che richiede di memorizzare, per ogni nodo, la lista dei predecessori — cioè il grafo con tutti gli archi invertiti, il *trasposto*. È quanto illustra la Figura 1: la riga  $v$  raccoglie gli archi *entranti* in  $v$ , non quelli uscenti. L'inversione non richiede un passo dedicato: costruendo il CSR si sceglie semplicemente il nodo di destinazione, anziché quello di origine, come indice di riga.

**Costo in memoria e modo di accesso.** La struttura occupa  $8(N+1) + 4M + 4N$  byte, ed è questa formula a produrre l'ultima colonna della Tabella 2: per **web-Google** 29,5 MB contro i 5,6 TB della matrice densa. Resta però un aspetto che tornerà nella Sezione 3.4. `row_ptr` e `col_idx` si leggono in ordine — accessi sequenziali, ideali per il prefetch del processore — mentre `contrib[col_idx[j]]` è un accesso casuale a un vettore che su questi grafi non entra in cache. Per ogni arco si legge un indice di 4 byte, si va a prendere un contributo in una posizione imprevedibile e si esegue una singola somma: il rapporto fra byte trasferiti e operazioni aritmetiche è bassissimo, ed è la ragione di fondo per cui il calcolo si rivelerà *memory-bound*.

## 2.2 Formato binario e pre-elaborazione

La conversione dal formato testuale SNAP al CSR avviene *una volta sola*, offline, tramite `tools/snap_to_csr.py`. Le motivazioni sono due:

1. l'analisi del testo è lenta e irregolare; eseguirla dentro il programma misurato significherebbe misurare il parser anziché l'algoritmo;
2. il codice C non ha così bisogno di alcun parser testuale, il che elimina un'intera classe di errori dalla parte valutata.

Gli identificatori SNAP sono sparsi (**web-Google** ha 875 713 nodi con id fino a 916 428) e vengono rimappati su un intervallo contiguo  $[0, N)$ ; la corrispondenza inversa è salvata in un file `.ids` separato, letto solo per riportare i risultati negli identificatori originali.

Il formato binario prodotto è il seguente:

| Campo            | Tipo     | Dimensione    |
|------------------|----------|---------------|
| magic "PRCSR001" | char[8]  | 8 byte        |
| n_nodes          | uint64   | 8 byte        |
| n_edges          | uint64   | 8 byte        |
| row_ptr          | uint64[] | $8(N+1)$ byte |
| col_idx          | uint32[] | $4M$ byte     |
| out_deg          | uint32[] | $4N$ byte     |

`col_idx` usa 32 bit anziché 64 perché gli elementi sono  $M$ : su `soc-LiveJournal1` la scelta dimezza l'occupazione da 552 MB a 276 MB, ed è sicura perché un indice di nodo è sempre minore di  $N < 2^{32}$ . Il caricamento lato C si riduce così a tre `fread`, senza alcuna conversione.

## 2.3 Struttura del codice

| Modulo                                        | Ruolo                                      | Righe |
|-----------------------------------------------|--------------------------------------------|-------|
| <code>tools/snap_to_csr.py</code>             | conversione SNAP $\rightarrow$ CSR binario | 173   |
| <code>src/csr.c</code> , <code>csr.h</code>   | caricamento e validazione                  | 293   |
| <code>src/pagerank.c</code> , <code>.h</code> | nucleo di calcolo                          | 186   |
| <code>src/main.c</code>                       | driver, output, log dei tempi              | 295   |
| <code>src/csr_info.c</code>                   | statistiche di un grafo convertito         | 91    |
| <code>tools/verify_pagerank.py</code>         | verifica indipendente                      | 325   |
| <code>tools/run_benchmarks.sh</code>          | campagna sperimentale                      | 60    |

## 2.4 Validazione del file in ingresso

Il caricatore verifica cinque proprietà prima di restituire il grafo, con altrettante funzioni dal nome esplicito:

- `check_header_counts` — i conteggi dichiarati sono compatibili con la dimensione del file;
- `check_header_size` — il file misura esattamente quanto l'intestazione implica;
- `check_rowptr_span` — gli offset coprono  $[0, M]$ ;
- `check_rowptr_monotonic` — gli offset non decrescono;
- `check_neighbours_are_existing_nodes` — ogni vicino memorizzato è un nodo esistente.

Due meritano una motivazione esplicita.

**Confronto fra intestazione e dimensione reale del file.** Prima di allocare alcunché si calcola quanti byte il file *dovrebbe* misurare secondo la propria intestazione e lo si confronta con la dimensione effettiva. Un file troncato o corrotto viene così respinto prima che i suoi numeri vengano usati per dimensionare un'allocazione.

**Ordine dei controlli come garanzia sintattica.** I due controlli sull'intestazione sono concatenati con `||`:

```

1 | if (check_header_counts(g, actual, path) != 0 ||
2 |    check_header_size(g, actual, path) != 0) {
3 |     goto fail;
4 | }
```

La valutazione *short-circuit* garantisce che il secondo controllo non venga mai eseguito su valori che il primo ha già respinto. Questo non è stilistico: è precisamente ciò che impedisce all'aritmetica di `check_header_size` di andare in overflow su un'intestazione corrotta che dichiari, ad esempio,  $2^{63}$  archi. Il vincolo d'ordine è così imposto dal linguaggio anziché essere affidato a un commento.

L'insieme dei controlli rende dimostrabile una proprietà usata dal nucleo di calcolo: il ciclo interno indicizza `contrib[col_idx[j]]` senza alcun controllo proprio di limiti, ed è sicuro solo perché l'invariante è stato stabilito una volta in fase di caricamento.

## 2.5 Implementazione sequenziale e OpenMP

Le due versioni condividono *lo stesso file sorgente*. Le direttive OpenMP sono ignorate dal compilatore quando non gli si passa `-fopenmp`, che produce quindi normali cicli seriali. Il vantaggio è duplice: non esiste duplicazione di codice che possa divergere, e soprattutto il riferimento sequenziale è *onesto*. Misurando la versione parallela limitata a un thread si otterrebbe un tempo peggiore (Tabella 3, colonna *1 thr*), perché si pagherebbe comunque la creazione del team di thread: usarla come riferimento gonfierebbe silenziosamente ogni speed-up dichiarato.

Il nucleo contiene due cicli paralleli per iterazione.

```
1 | #pragma omp parallel for schedule(static) reduction(+ : dangling)
2 | for (v = 0; v < n; v++) {
3 |     if (g->out_deg[v] == 0) {
4 |         contrib[v] = 0;
5 |         dangling += cur[v];
6 |     } else {
7 |         contrib[v] = cur[v] / g->out_deg[v];
8 |     }
9 | }
```

Listing 1: Calcolo dei contributi e massa dangling.

La clausola `reduction(+ : dangling)` assegna a ogni thread una copia privata della variabile, inizializzata a zero, e ne somma i valori al termine del ciclo. È la soluzione corretta e veloce al problema dell'aggiornamento concorrente: una variabile condivisa protetta da lock serializzerebbe l'intero ciclo. Lo scheduling è `static` perché il lavoro per iterazione è costante.

```
1 | #pragma omp parallel for schedule(dynamic, 256) reduction(+ : error)
2 | for (v = 0; v < n; v++) {
3 |     accum_t sum = 0.0;
4 |     for (j = g->row_ptr[v]; j < g->row_ptr[v + 1]; j++)
5 |         sum += contrib[g->col_idx[j]];
6 |     nxt[v] = base + d * sum;
7 |     error += fabs(nxt[v] - cur[v]);
8 | }
```

Listing 2: Accumulo dai predecessori (CSR).

Qui lo scheduling è `dynamic`: la lunghezza delle righe varia di ordini di grandezza (su `web-BerkStan` da 0 a 84208 elementi), quindi una suddivisione statica assegnerebbe a ogni thread lo stesso *numero di righe* ma non la stessa *quantità di lavoro*, lasciando inattivi i thread che hanno ricevuto solo nodi di grado basso. La dimensione del blocco (256) è un parametro di tuning.

**Alternanza dei buffer.** L'iterazione alterna due vettori scambiando i puntatori anziché copiare i dati; una sola `memcpy` finale è necessaria, e solo se il numero di scambi è dispari.

## 2.6 Precisione: float contro double

Il tipo dei rank è definito da un unico `typedef` commutabile in fase di compilazione con `-DPAGERANK_FLOAT`. La motivazione è che le GPU di fascia consumer eseguono l'aritmetica FP64 a una frazione ridotta del ritmo FP32, per cui il kernel CUDA richiederà verosimilmente `float`; mantenere entrambe le varianti raggiungibili dallo stesso sorgente permette di confrontarne accuratezza e velocità.

Gli accumulatori restano però sempre in `double`: memorizzare milioni di rank in singola precisione dimezza il traffico di memoria, ma *sommarli* in singola precisione perderebbe molta più accuratezza di quanta se ne guadagni.

## 2.7 Implementazione ibrida OpenMP + CUDA

**[DA COMPLETARE]** Sezione da scrivere interamente. Dovrà documentare: partizionamento del carico fra  $V_{host}$  e  $V_{device}$  in base al grado dei nodi; orchestrazione con stream CUDA non predefiniti e `cudaMemcpyAsync` per sovrapporre trasferimenti e calcolo; combinazione dell'errore di convergenza calcolato su CPU e GPU; dimensionamento di griglia e blocchi a runtime tramite `cudaGetDeviceProperties` e `cudaMemGetInfo`, come richiesto dal requisito di portabilità; granularità adattiva (un thread, un warp o un blocco per riga a seconda della sua lunghezza).

**Dati già disponibili per il progetto del kernel.** La distribuzione delle lunghezze di riga, misurata da `csr_info`, è riportata in Tabella 1: fra il 45% e il 64% delle righe contiene al più 4 elementi, mentre singole righe raggiungono decine di migliaia. Questo giustifica quantitativamente uno schema a granularità variabile anziché un kernel monolitico con un thread per riga.

Tabella 1: Distribuzione delle lunghezze di riga (predecessori per nodo).

| Grafo            | 1–4    | 5–32   | > 32   | max    |
|------------------|--------|--------|--------|--------|
| wiki-Vote        | 3,0 %  | 12,9 % | 17,6 % | 457    |
| web-Google       | 56,7 % | 22,3 % | 2,6 %  | 6 326  |
| web-BerkStan     | 64,2 % | 22,6 % | 3,3 %  | 84 208 |
| soc-LiveJournal1 | 45,2 % | 36,7 % | 10,7 % | 13 906 |

## 2.8 Strategia di verifica

Il programma produce fino a 4,8 milioni di valori, non ispezionabili a occhio. La correttezza è quindi stabilita ricalcolando il risultato per vie che non condividono codice con l'implementazione sotto esame (`tools/verify_pagerank.py`):

1. il file CSR binario viene confrontato con una rilettura indipendente dell'edge list, effettuata in Python puro;
2. PageRank viene ricalcolato su una **matrice densa**  $N \times N$  mediante iterazione di potenza: nessun CSR, nessun ciclo di accumulo, quindi un errore nella rappresentazione sparsa non può sopravvivere al confronto;
3. il riferimento denso viene a sua volta confrontato con **NetworkX**, implementazione di terze parti. Questo passo esclude una classe di errori che i primi due non coprono: il riferimento denso è indipendente nel metodo ma scritto dallo stesso autore del codice in esame, quindi un fraintendimento dell'algoritmo (tipicamente la convenzione sui nodi dangling) potrebbe essere riprodotto identico in entrambi;
4. si verifica infine che il binario C sia *giunto a convergenza*, che il ranking e i valori coincidano e che la somma dei rank sia 1.

Il controllo di convergenza al punto 4 non è ridondante: forzando l'arresto dopo 5 iterazioni il *ranking* dei primi risultati è ancora corretto mentre i valori sono lontani dall'essersi stabilizzati, quindi verificare solo l'ordinamento accetterebbe un risultato quantitativamente sbagliato.

Lo strumento di verifica è stato a sua volta collaudato iniettando un guasto: ricostruendo il CSR da un grafo privato di 457 archi entranti in un nodo, la verifica fallisce a ogni livello,

mentre il controllo sulla somma dei rank resta correttamente silenzioso perché il grafo alterato è internamente coerente.

Poiché la matrice densa richiede  $N^2$  elementi, questa verifica è possibile solo sul grafo più piccolo: 386 MB per **wiki-Vote** contro i 5,6 TB che servirebbero per **web-Google**. È questa la ragione per cui **wiki-Vote** fa parte dell'insieme di test.

### 3 Risultati sperimentali

#### 3.1 Ambiente di prova

|                   |                                                                 |
|-------------------|-----------------------------------------------------------------|
| Sistema operativo | Ubuntu 22.04.5 LTS, kernel Linux 6.8                            |
| CPU               | Intel Core i5-9300H, 4 core fisici / 8 thread, 2,40–4,10 GHz    |
| Cache             | L1d 128 KiB, L2 1 MiB, L3 8 MiB                                 |
| RAM               | 15 GiB                                                          |
| GPU               | NVIDIA GeForce GTX 1050 Ti, 4 GiB, compute capability 6.1, 6 SM |
| Compilatore       | gcc, OpenMP 4.5; CUDA Toolkit 12.8                              |
| Opzioni           | -std=c11 -Wall -Wextra -O2                                      |

Tutti i binari compilano senza alcun avviso.

#### 3.2 Istanze di prova

Sono stati usati quattro grafi reali della collezione SNAP [3], scelti in modo da coprire circa tre ordini di grandezza nel numero di archi (fattore  $670\times$  fra il minimo e il massimo).

Tabella 2: Istanze utilizzate.

| Grafo            | Nodi      | Archi      | Dangling | max in-deg | CSR      |
|------------------|-----------|------------|----------|------------|----------|
| wiki-Vote        | 7 115     | 103 689    | 1 005    | 457        | 0,5 MB   |
| web-Google       | 875 713   | 5 105 039  | 136 259  | 6 326      | 29,5 MB  |
| web-BerkStan     | 685 230   | 7 600 595  | 4 744    | 84 208     | 36,8 MB  |
| soc-LiveJournal1 | 4 847 571 | 68 993 773 | 539 119  | 13 906     | 318,7 MB |

I due grafi web sollecitano aspetti diversi: **web-BerkStan** ha un singolo *hub* estremo (84 208 archi entranti), mentre **soc-LiveJournal1** ha la distribuzione più pesante nella fascia inter-media (Tabella 1).

#### 3.3 Correttezza

| Confronto                   | Accordo              | Esclude                               |
|-----------------------------|----------------------|---------------------------------------|
| C contro riferimento denso  | $4,8 \cdot 10^{-10}$ | errori in CSR o nel ciclo di accumulo |
| NetworkX contro denso       | $4,5 \cdot 10^{-12}$ | fraintendimenti dell'algoritmo        |
| OpenMP contro sequenziale   | $2,1 \cdot 10^{-14}$ | race condition                        |
| float contro double         | $2,2 \cdot 10^{-7}$  | perdita di precisione anomala         |
| Somma dei rank (ogni prova) | 1,000000             | dispersione della massa dangling      |

Lo scarto relativo fra OpenMP e sequenziale è dovuto unicamente al diverso ordine di somma introdotto dalla riduzione. Lo scarto della versione **float** coincide con l'epsilon di macchina della singola precisione ( $1,19 \cdot 10^{-7}$ ): non si perde quindi nulla oltre al limite intrinseco del formato.

### 3.4 Scalabilità della versione OpenMP

Ogni configurazione è stata eseguita tre volte; si riporta il *minimo*, perché l'attività di sistema può solo rallentare un'esecuzione, mai accelerarla, e la prova più rapida è quindi la meno disturbata.

Tabella 3: Tempi di esecuzione in secondi (minimo su 3 ripetizioni).

| Grafo            | seq    | 1 thr  | 2 thr | 4 thr | 8 thr | float 8 | speed-up |
|------------------|--------|--------|-------|-------|-------|---------|----------|
| web-Google       | 1,326  | 1,353  | 0,741 | 0,459 | 0,519 | 0,347   | 2,89×    |
| web-BerkStan     | 0,715  | 0,712  | 0,400 | 0,254 | 0,269 | 0,214   | 2,82×    |
| soc-LiveJournal1 | 11,863 | 12,442 | 7,457 | 5,088 | 5,014 | 3,608   | 2,37×    |

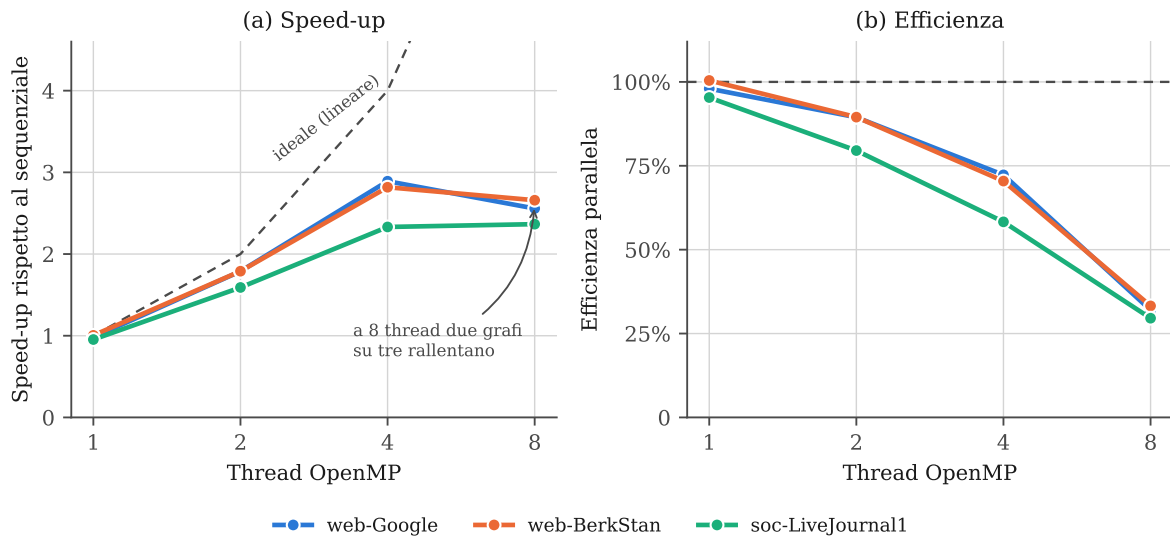


Figura 2: Scalabilità della versione OpenMP. Il riferimento per lo speed-up è la build sequenziale vera, compilata senza `-fopenmp`. La linea tratteggiata in (a) indica lo speed-up lineare ideale; in (b) l'efficienza del 100 %. Su `web-Google` e `web-BerkStan` il passaggio da 4 a 8 thread *peggiora* il tempo di esecuzione. Figura generata da `tools/plot_results.py` a partire da `results/bench.csv`.

La Figura 2 rende visibile il fenomeno descritto sopra: le tre curve si allontanano dall'andamento lineare già a 2 thread e si appiattiscono fra 4 e 8, con una vera e propria inversione su due grafi su tre. L'efficienza scende dal 90 % circa a 2 thread al 70 % a 4, fino a circa il 30 % a 8: i thread aggiuntivi non trovano banda di memoria disponibile.

**Osservazione principale: il calcolo è memory-bound.** Su due grafi su tre l'esecuzione con 8 thread è *più lenta* di quella con 4. Il processore possiede 4 core fisici presentati come 8 thread logici; i due thread che condividono un core condividono anche la via di accesso alla memoria. Poiché il ciclo interno passa la maggior parte del tempo in attesa di letture sparse su `contrib`, i thread aggiuntivi producono contesa anziché capacità di calcolo.

L'efficienza parallela su 4 core è del 60–72%. Questo risultato è il dato più importante dell'analisi: identifica il collo di bottiglia nella *larghezza di banda della memoria*, non nella potenza di calcolo, e costituisce la motivazione quantitativa per il passaggio alla GPU — il cui vantaggio non è tanto il maggior numero di core quanto la banda di memoria sensibilmente superiore.

**Nota sull'istanza più piccola.** I tempi di `wiki-Vote` non sono riportati in Tabella 3: con durate dell'ordine del millesimo di secondo la dispersione fra ripetizioni della stessa configurazione raggiunge un fattore  $51\times$ , il che significa che si misurerebbe l'avvio del team di thread e non l'algoritmo. Il grafo resta nell'insieme di prova con il ruolo di riferimento di correttezza (Sezione 2.8).

### 3.5 Confronto fra le varianti `float` e `double`

La variante in singola precisione è più rapida del 32% su `web-Google` (0,347 s contro 0,459 s), a parità di numero di iterazioni e con ranking identico. Il guadagno è coerente con il dimezzamento dei byte trasferiti per ogni rank, ed è un'ulteriore conferma indiretta della natura memory-bound del calcolo.

Il limite della singola precisione si manifesta sulla soglia di convergenza: su `web-Google` l'errore  $L_1$  non scende sotto  $\approx 5 \cdot 10^{-9}$ , per cui una tolleranza di  $10^{-9}$  non viene mai raggiunta mentre la doppia precisione converge in 104 iterazioni. Alla tolleranza utilizzata negli esperimenti ( $10^{-6}$ ) entrambe convergono nello stesso numero di iterazioni.

### 3.6 Risultati della versione ibrida

**[DA COMPLETARE]** Sezione da scrivere: tempi della versione CUDA e ibrida, confronto con OpenMP, occupancy, effetto della granularità adattiva, effetto della sovrapposizione fra trasferimenti e calcolo.

## 4 Valutazioni e osservazioni

### 4.1 Punti di forza

- **Assenza di sincronizzazione nel nucleo di calcolo.** La formulazione *pull*, con il grafo memorizzato trasposto, elimina alla radice le scritture concorrenti, e si trasferisce invariata alla versione CUDA.
- **Riferimento sequenziale onesto.** Sequenziale e parallelo provengono dallo stesso sorgente, il che rende impossibile una divergenza fra le due versioni e non gonfia lo speed-up dichiarato.
- **Verifica indipendente e collaudata.** Tre implementazioni concordano, e lo strumento di verifica è stato validato con un'iniezione di guasto.
- **Riproducibilità.** L'intera campagna sperimentale è riproducibile con un solo comando.

### 4.2 Punti deboli e limiti

- **Il convertitore non scala oltre le istanze usate.** Il convertitore Python mantiene l'intera edge list in memoria, con un costo di circa 100 byte per arco (6,66 GB misurati su `soc-LiveJournal1`). Un'istanza da un miliardo di archi richiederebbe un convertitore in C con ordinamento esterno.
- **Formato binario non standard.** Il formato `.csr` è specifico del progetto: è velocissimo da caricare, ma non ispezionabile con strumenti esterni. La scelta è stata fatta consapevolmente, privilegiando l'assenza di parsing nel percorso misurato.
- **Dimensione della porzione dinamica non ottimizzata.** Il valore 256 di `schedule(dynamic, 256)` non è stato oggetto di una ricerca sistematica.

### 4.3 Possibili miglioramenti

- **Gestione out-of-core.** Era prevista in fase di proposta una logica di streaming per grafi eccedenti la memoria della GPU. Le misure hanno mostrato che non è necessaria per le istanze considerate: il grafo più grande occupa 356 MB contro i 4 GB disponibili, e servirebbe un'istanza dell'ordine del miliardo di archi per superare la soglia. Resta un possibile sviluppo futuro, subordinato però alla riscrittura del convertitore.
- **Granularità adattiva sulla GPU.** Le misure di Tabella 1 sono già disponibili ma non ancora sfruttate da alcun kernel.
- **Reciproci del grado precalcolati.** Il ciclo dei contributi esegue  $N$  divisioni per iterazione; un array `inv_out_deg[N]` con  $1/\text{outdeg}(u)$  calcolato una volta sola le ridurrebbe ad altrettante moltiplicazioni. Su CPU il guadagno atteso è piccolo — quel ciclo tocca  $N$  elementi contro gli  $M$  del ciclo di accumulo, e il calcolo è comunque *memory-bound* — e in doppia precisione l'array costerebbe  $8N$  byte contro i  $4N$  di `out_deg`, raddoppiando il traffico proprio dove sta il collo di bottiglia. Vale invece la pena provarlo sulla GPU e nella variante `float`, dove il rapporto di costo fra divisione e moltiplicazione è più sfavorevole e l'occupazione resta invariata. Da verificare l'effetto sulla precisione:  $x \cdot (1/d)$  introduce un arrotondamento in più rispetto a  $x/d$ .
- **Ricerca sui parametri di scheduling** (politica e dimensione del blocco) e confronto con `guided`.

**[DA COMPLETARE]** Aggiungere la valutazione personale complessiva una volta completata la fase CUDA.

## 5 Guida all'uso e riproducibilità

### 5.1 Requisiti

gcc con supporto OpenMP; CUDA Toolkit per la versione ibrida; Python 3 con `numpy` per la conversione dei dataset, più `networkx` e `scipy` per la sola verifica di correttezza.

### 5.2 Compilazione

```
| make
```

Produce quattro eseguibili in `build/`: `pagerank_seq` (senza OpenMP), `pagerank_omp`, `pagerank_omp_float` e `csr_info`. Tutti derivano dagli stessi file sorgente.

### 5.3 Preparazione dei dati

```
| mkdir -p data/snap && cd data/snap
| for g in wiki-Vote web-Google web-BerkStan soc-LiveJournal1; do
|     curl -O https://snap.stanford.edu/data/$g.txt.gz && gunzip $g.txt.gz
| done
| cd ../..
| python3 tools/snap_to_csr.py data/snap/web-Google.txt
```

## 5.4 Esecuzione

```
| ./build/pagerank_omp data/snap/web-Google.csr -i data/snap/web-Google.ids
```

Opzioni: `-d` damping, `-t` tolleranza, `-n` iterazioni massime, `-k` quanti nodi stampare, `-o` salva tutti i rank, `-c` accoda una riga CSV con i dati dell'esecuzione. Il numero di thread si imposta con `OMP_NUM_THREADS`.

## 5.5 Verifica della correttezza

```
| python3 tools/verify_pagerank.py data/snap/wiki-Vote.txt
```

Termina con stato diverso da zero in caso di discrepanza, e può quindi essere usato come test di regressione.

## 5.6 Riproduzione degli esperimenti

```
| tools/run_benchmarks.sh
```

Esegue ogni grafo con le varianti sequenziale, OpenMP (1/2/4/8 thread) e float, con tre ripetizioni, e scrive `results/bench.csv` (una riga per esecuzione) e i vettori dei rank completi. Le impostazioni sono sovrascrivibili da ambiente, ad esempio `GRAPHS="wiki-Vote" REPS=1 tools/run_benchmarks.sh`.

## 5.7 Codice sorgente

Il codice completo, con la cronologia di sviluppo, è disponibile all'indirizzo

<https://github.com/EmanueleCecchia/pagerank-openmp-cuda>

Il repository contiene i sorgenti C, gli strumenti Python di conversione e verifica, lo script della campagna sperimentale e il file `results/bench.csv` con i dati grezzi di tutte le esecuzioni. I dataset e i vettori dei rank non sono versionati per ragioni di dimensione: i primi si ricaricano con i comandi della Sezione 5.3, i secondi si rigenerano con `tools/run_benchmarks.sh`.

## Riferimenti bibliografici

- [1] L. Page, S. Brin, R. Motwani, T. Winograd, *The PageRank Citation Ranking: Bringing Order to the Web*, Stanford InfoLab, 1999.
- [2] V. Visali, R. Manimegalai, S. Noor Mahammad, A. Sunitha Nandhini, *Performance Analysis of Parallelized PageRank Algorithm using OpenMP, MPI and CUDA*, ICSSEECC, pp. 44–49, IEEE, 2024.
- [3] J. Leskovec, A. Krevl, *SNAP Datasets: Stanford Large Network Dataset Collection*, <https://snap.stanford.edu/data>, 2014.
- [4] D. B. Kirk, W. W. Hwu, *Programming Massively Parallel Processors*, 3rd ed., Morgan Kaufmann, 2016.
- [5] NVIDIA, *CUDA C++ Programming Guide*.